SNLP COURSE PROJECT 2026

Handwritten Exam Digitizer

Turning photos of handwritten answer sheets into clean, structured digital text

Project document: pipeline, tech stack, web UI and contributions

Group 1

Roll No.	Name
E001	Mohammad Kaif
E002	Arnav Mhatre
E004	Ojas Sharma
E005	Madhav Patel
E007	Aayush Nayak

AI University: Building Reusable NLP Assets

Contents

1. Project at a glance 4
The problem 4
Our solution 4
Key results 4
2. The end-to-end pipeline 5
Each step in one line 5
The output record 6
3. Cleaning the image (Week 7 OCR front end) 7
The key discovery (Week 5) 8
Things we tried and rejected 8
4. Finding the key details (NER) 9
Two versions 9
The look-alike table 10
5. How we measure accuracy 11
Results per page 11
6. Tech stack 12
Core pipeline 12
Web app 12
7. Web UI features 14
The five tabs 14
Options on the Digitize tab 14
While it runs and after 14
What happens behind the scenes 15
8. How the project grew, week by week 16
9. Code structure 17
10. How to run it 18
11. Known limits and next steps 18
12. Contributions 19
E001 Mohammad Kaif 20
E002 Arnav Mhatre 21
E004 Ojas Sharma 22
E005 Madhav Patel 23
E007 Aayush Nayak 24
13. Glossary 26

> **How to read this document**
>
> Every section starts with an **In short** box. Read only those boxes for a 2-minute overview.
>
> Flowcharts show how data moves. Tables hold the details. Technical file names are shown like `this`.
>
> Section 12 (Contributions) explains each member's work in simple words they can present.

1. Project at a glance

> **In short**
>
> We built a tool that takes a **photo or scan of a handwritten answer sheet** and gives back **typed text** plus key details like **student ID, question number and marks**, in a clean JSON format.
>
> It runs as a Python function anyone can reuse, and as a **web app** you open in the browser.

The problem

- Exam answer sheets are handwritten. Checking, storing or searching them means someone has to read and type everything by hand.
- Phone photos of notebook pages are messy: ruled lines, shadows, tilted pages, pink margins.
- Normal OCR (text reading software) is built for printed text, so it makes many mistakes on handwriting.

Our solution

- Clean the photo so the handwriting stands out and the ruled lines disappear.
- Read it with the Tesseract OCR engine.
- Use NLP (Named Entity Recognition) to find the student ID, question number, marks and page number, even when OCR misspells them.
- Package everything into one reusable function, `digitize_page()`, and a simple web UI.

Key results

What we measured	Before	After (Week 7)	Meaning
Character Error Rate (CER), average	0.426	0.189	Wrong letters fell from ~43% to ~19%
CER on the neater page	0.397	0.037	Only ~4 wrong letters in every 100
Word Error Rate (WER), average	0.727	0.566	Fewer wrong words
NER F1 on real OCR mistakes	0.308	1.0	Details found correctly on our test cases
Time per page	-	~1 second	Fast enough for whole answer booklets

Lower CER / WER is better. NER F1 is between 0 and 1, higher is better.

2. The end-to-end pipeline

> **In short**
> Upload → turn PDF into images → clean the image → read the handwriting → tidy the text → find the key details → output a JSON record.

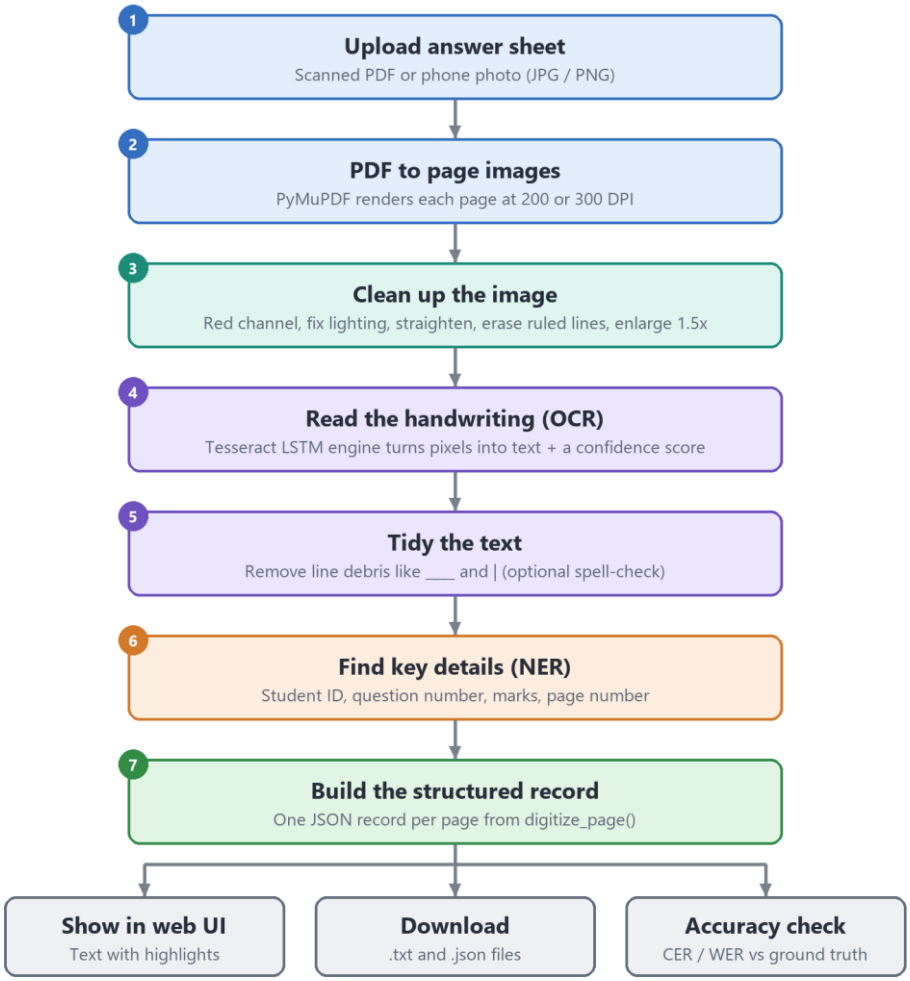


Figure 1. The full pipeline, from upload to output

Each step in one line

#	Step	What happens (simple words)	Tool	File
1	Upload	User gives a scanned PDF or a phone photo.	Web UI	`app/server.py`
2	PDF to images	Each PDF page becomes a picture (200 or 300 DPI).	PyMuPDF	`app/server.py`
3	Clean the image	Remove ruled lines, shadows and tilt so only ink remains.	OpenCV, NumPy	`src/ocr_pipeline.py`

#	Step	What happens (simple words)	Tool	File
4	OCR	Tesseract reads the picture and gives text + confidence.	Tesseract, pytesseract	`src/ocr_pipeline.py`
5	Tidy text	Delete junk like ____ or | left by lines. Spell-check is optional.	regex, pyspellchecker	`src/ocr_pipeline.py`
6	NER	Find student ID, question no., marks, page no.	regex rules / spaCy	`src/ner_improved.py`
7	Record	Put everything into one JSON dictionary.	Python	`src/digitizer.py`

The output record

Every page comes out as one record like this. Another program (for example Group 2's AI Exam Grader) can use it directly.

Field	Example	Meaning
student_id	E007	Best guess of the student's roll number
question	Q1	Best guess of the question number
answer_text	OSI Model in Email…	The full text read from the page
page	1	Page number (passed in by the caller)
marks_detected	7/10	Marks written on the page, if any
ocr_confidence	0.54	How sure Tesseract was (0 to 1). Low = ask a human to check
entities_found	[…]	Every candidate found, kept for manual review

> **How someone else uses it**
>
> ```
> from digitizer import digitize_page
> result = digitize_page("scan.jpg", page_number=4)
> ```
>
> That is all. The caller never needs to know about OCR or NER.

3. Cleaning the image (Week 7 OCR front end)

In short

Phone photos of ruled pages confuse OCR. We remove the lines, shadows and tilt **without** turning the image into harsh black and white, then enlarge it. This alone cut errors by more than half.

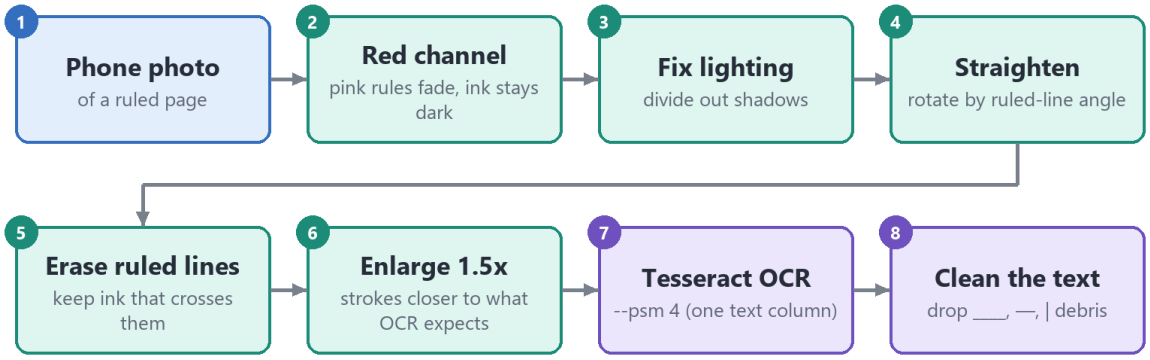


Figure 2. The eight steps inside the OCR front end

Step	What it does	Why it helps
Red channel	Keep only the red colour layer of the photo.	Pink/red margins and rules almost vanish; blue and black ink stay dark.
Fix lighting	Divide the image by a blurred copy of itself.	Removes shadows and uneven phone lighting.
Straighten	Measure the angle of the ruled lines and rotate.	Tesseract reads straight lines much better.
Erase ruled lines	Paint faint line pixels white, keep ink that crosses them.	Lines no longer merge with letters.
Enlarge 1.5x	Upscale the page.	Letters become the size Tesseract's model expects.
Tesseract (psm 4)	Read the page as one column of text.	Best mode for answer sheets in our tests.
Clean text	Remove ____, —, | debris.	Leftover line marks are not real text.
Spelling (optional)	Fix misspelt lower-case words.	Off by default: it would also 'fix' the student's own mistakes.

The key discovery (Week 5)

> **Old cleaning made things worse**
>
> The Week 1-2 cleaning used **adaptive thresholding** (turning everything pure black or white). On photographed ruled pages this made the ruled lines thick and dark, so they **stuck to the letters** and OCR got worse.
>
> This finding is why the Week 7 front end avoids hard black/white conversion.

Things we tried and rejected

Idea	Result	Decision
Microsoft TrOCR (transformer for handwriting)	Similar accuracy, but 3-5 minutes per page on CPU	Too slow
Reading line by line (`--psm 7`)	Worse (CER 0.21+ on the neat page)	Rejected
Tesseract `tessdata_best` model	No better, twice as slow	Rejected

4. Finding the key details (NER)

> **In short**
>
> **NER (Named Entity Recognition)** means finding important pieces of text and labelling them. We look for 4 labels: **STUDENT_ID, QUESTION_NUM, MARKS, PAGE_NUM**.
>
> Version 1 used spaCy rules. Version 2 also understands OCR mistakes like reading 0 as O.

Two versions

	Version 1: spaCy rules (Week 3)	Version 2: OCR-noise tolerant (Week 4 + 7)
How	spaCy EntityRuler with regex patterns on each token	Custom regex finders + a 'look-alike' fixing table
Handles OCR mistakes?	No. '23DOO1' is missed	Yes. '23DOO1' becomes 23D001
Extra tricks	-	ID after 'Roll no' / 'ID'; 'QL.' and '@1)' read as Q1; dates not taken as marks
Keeps all candidates?	No	Yes, for human review
F1 on real OCR cases	0.308	1.0

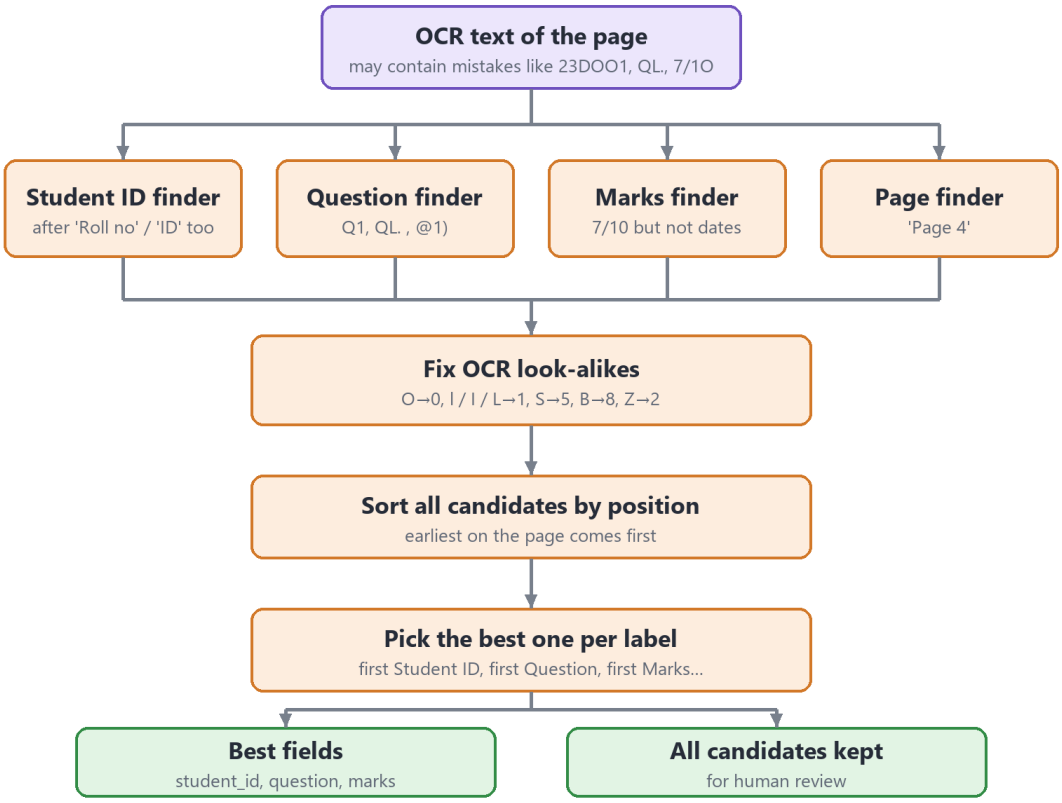


Figure 3. How the improved NER picks the details

The look-alike table

OCR often confuses letters and digits that look similar. Inside an ID, question or marks, we swap them back:

OCR read	O / o / D	I / l / L / |	S	B	Z
We use	0	1	5	8	2

> **Example**
>
> OCR text: `23DOO1 Q3 The Viterbi algorithm … Page 4 Marks 7/1O`
>
> Result: student_id = **23D001**, question = **Q3**, marks = **7/10**, page = **Page 4**

5. How we measure accuracy

> **In short**
> We typed out the real text of each page by hand (**ground truth**) and compared it with what OCR produced. Fewer differences = better.

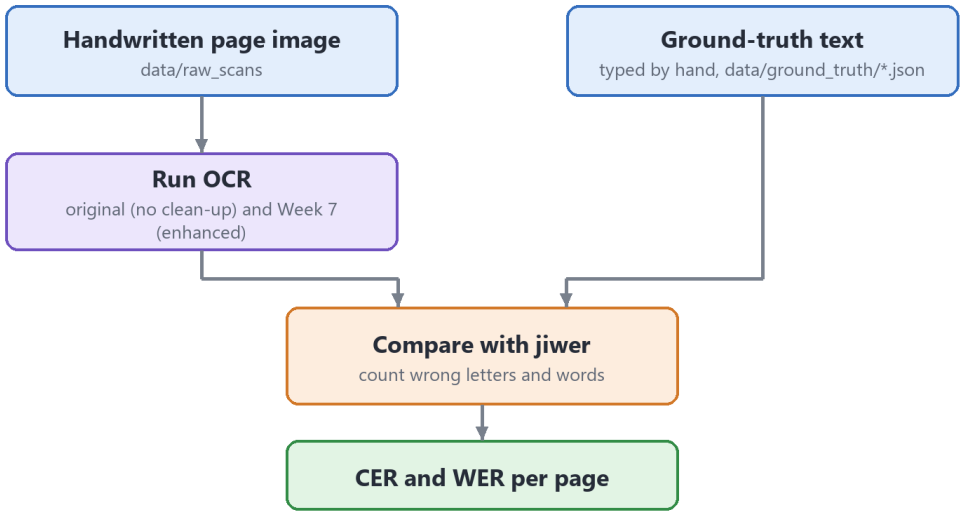


Figure 4. Evaluation flow

Metric	Simple meaning	Good value
CER (Character Error Rate)	Out of all letters, how many are wrong, missing or extra	Close to 0
WER (Word Error Rate)	Same idea, counted per word	Close to 0
Precision (NER)	Of the details we found, how many were right	Close to 1
Recall (NER)	Of the real details, how many we found	Close to 1
F1 (NER)	One number that balances precision and recall	Close to 1

Results per page

Page	Original OCR (CER / WER)	Week 7 (CER / WER)
aayush_page-01 (neat)	0.397 / 0.490	0.037 / 0.163
rish_page-1 (cursive)	0.455 / 0.964	0.341 / 0.969
Average	0.426 / 0.727	0.189 / 0.566

Run `python src/evaluate.py` to reproduce both columns.

6. Tech stack

> **In short**
> Everything is **Python** on the back end and plain **HTML/CSS/JavaScript** on the front end. All tools are free and open source and run on a normal laptop (no GPU needed).

Core pipeline

Tool	What it is	What we use it for
Python 3	Programming language	All pipeline and server code
OpenCV (opencv-python)	Image processing library	Colour channel, lighting fix, rotation, line removal, resizing
NumPy	Fast number arrays	Pixel maths inside the image cleaning
Tesseract OCR	Google's open-source OCR engine (LSTM based)	Reading handwriting from the cleaned image
pytesseract	Python wrapper for Tesseract	Calling Tesseract and getting word confidences
Pillow	Python image library	Opening images for OCR
spaCy (en_core_web_sm)	NLP library	Week 3 rule-based NER using EntityRuler
Python `re` (regex)	Pattern matching	Improved NER finders and text clean-up
pyspellchecker	Dictionary spell checker	Optional spelling correction
jiwer	Speech/OCR metrics library	Calculating CER and WER

Web app

Tool	What it is	What we use it for
Flask	Small Python web framework	Server, upload API, job status API, file viewer
PyMuPDF (fitz)	PDF library	Turning each PDF page into an image
HTML / CSS / JavaScript	Web page building blocks	The five-tab user interface
Mermaid	Diagrams from text	Flowcharts tab in the UI
highlight.js, marked	Code colouring, Markdown	Code viewer in the Architecture tab

Tool	What it is	What we use it for
Windows .bat scripts	Double-click scripts	`start.bat` / `stop.bat` to start and stop the app
Git + GitHub	Version control	Sharing and tracking code across the group

7. Web UI features

> **In short**
> Double-click `start.bat`, the browser opens at http://127.0.0.1:5000. Drop a PDF or photo, click Run, watch each step live, then read, check and download the text.

The five tabs

Tab	What you can do
Digitize	Upload a PDF/photo or pick a sample, choose options, run the pipeline, see results
How it works	Each pipeline step in plain language plus a week-by-week project timeline
Flowcharts	End-to-end flow, inside digitize_page(), NER logic, web request flow, evaluation flow
Tech stack	What each library is and what it does in this project
Architecture	Folder tree with descriptions and an in-browser code viewer

Options on the Digitize tab

Option	Choices	Why it exists
NER mode	Improved (Week 4) / spaCy rules (Week 3)	Compare both NER versions live
Image enhancement	Smart (Week 7) / None / Threshold (Week 1-2)	Show how cleaning changes accuracy
Spell correction	On / Off	Off by default to keep the student's own words
Render quality	Fast 200 DPI / Best 300 DPI	Speed vs. detail for PDFs

While it runs and after

- **Live pipeline steps**, a progress bar and an activity log with timings for every page.
- **Side-by-side view**: the original scan vs. 'What OCR saw' (the cleaned image), with zoom.
- **Page navigation** for multi-page PDFs (up to 40 pages).
- **Output sub-tabs**: Text (with entities highlighted), Detected details, JSON record, Accuracy check.
- **Accuracy check**: CER/WER appear automatically if ground truth exists, or paste your own true text.
- **Copy and download**: copy page / copy all, download `.txt` and `.json`.
- **Extras**: light/dark theme, OCR health indicator, and a power button to shut the server down.

What happens behind the scenes

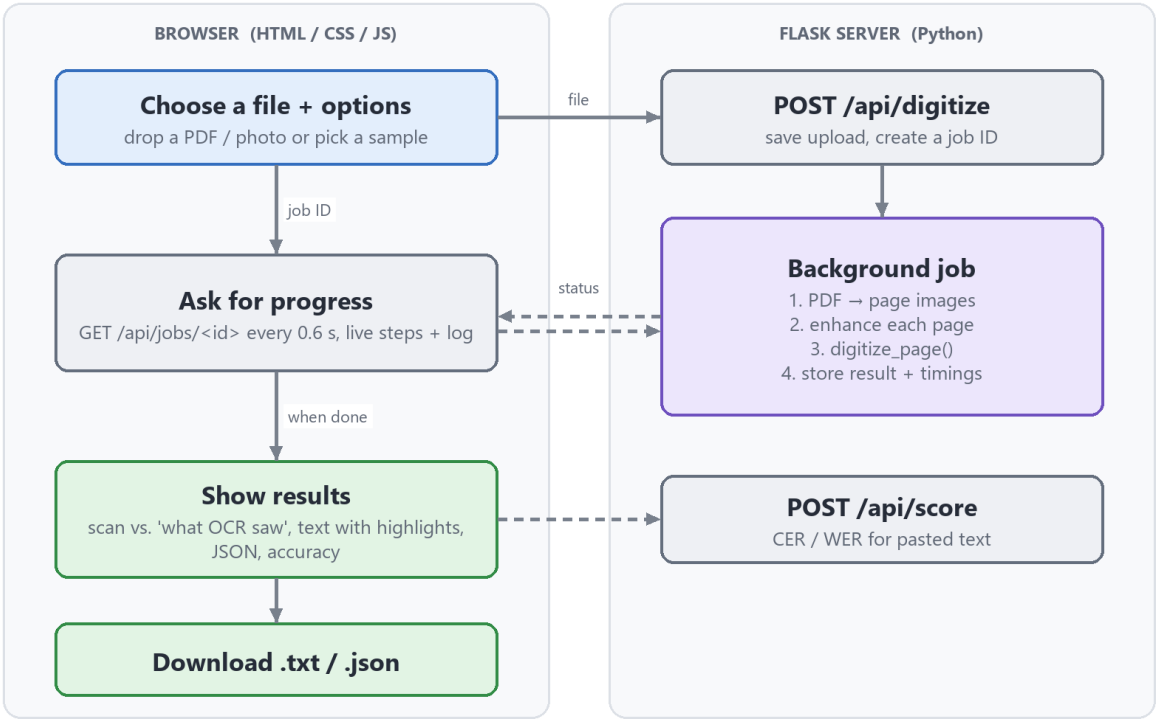


Figure 5. Browser and server talking during one run

The server does the heavy work in a **background job** so the page never freezes. The browser asks for progress every 0.6 seconds and shows each finished step.

8. How the project grew, week by week

> **In short**
> We started with a simple baseline, tested it on real pages, found what was going wrong, and fixed it step by step.

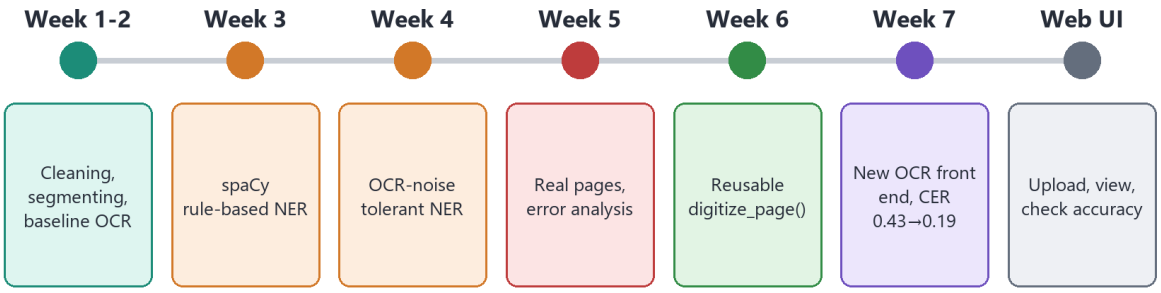


Figure 6. Project timeline

Week	What we did	Main files
1-2	Image cleaning, question splitting, Tesseract baseline	`preprocess.py`, `segment.py`, `baseline_ocr.py`
3	Rule-based NER with spaCy, first NER scores	`ner_extractor.py`, `process_with_ner.py`, `evaluate_ner.py`
4	NER that tolerates OCR mistakes, comparison with Week 3	`ner_improved.py`, `compare_ner_approaches.py`
5	Ran on real handwritten pages, CER/WER, error analysis	`evaluate.py`, `data/ground_truth/`
6	Wrapped everything as one reusable function	`digitizer.py`
7	New OCR front end, NER fixes for real OCR errors	`ocr_pipeline.py`, `ner_improved.py`
UI	Web app with five tabs, start/stop scripts	`app/`, `start.bat`, `stop.bat`

9. Code structure

> **In short**
> `src/` holds the pipeline, `app/` holds the web UI, `data/` holds images and ground truth.

Path	What's inside
`data/raw_scans/`	Original photos / scans of answer sheets
`data/preprocessed/`	Cleaned images (Week 1-2 method)
`data/ground_truth/`	Hand-typed true text, one JSON per page
`src/preprocess.py`	Week 1-2 cleaning: grayscale, blur, adaptive threshold, deskew
`src/segment.py`	Splits page text into questions using Q1, Q2… markers
`src/baseline_ocr.py`	Plain Tesseract reading + average confidence
`src/ner_extractor.py`	spaCy rule-based NER
`src/ner_improved.py`	OCR-noise tolerant NER
`src/compare_ner_approaches.py`	Scores both NER versions side by side
`src/evaluate.py`, `src/evaluate_ner.py`	CER/WER and NER precision/recall/F1
`src/ocr_pipeline.py`	Week 7 image enhancement + text clean-up
`src/digitizer.py`	The reusable `digitize_page()` function
`app/server.py`	Flask server: upload, jobs, scoring, file viewer
`app/templates/`, `app/static/`	HTML, CSS and JavaScript of the UI

10. How to run it

1. Install Python packages: `pip install -r requirements.txt`
2. Download the spaCy model: `python -m spacy download en_core_web_sm`
3. Install Tesseract (Windows): `winget install UB-Mannheim.TesseractOCR`
4. Double-click `start.bat` (the browser opens by itself). Use `stop.bat` or the power button to stop.
5. To check accuracy from the command line: `cd src` then `python evaluate.py`

> **Note**
> The UI loads fonts, diagrams and code colouring from the internet, so those parts need a connection. The pipeline itself runs fully offline.

11. Known limits and next steps

- Very cursive handwriting is still hard (CER 0.34 on the cursive page).
- Tesseract sometimes misreads handwritten IDs completely (E007 → 'Eool'), so `student_id` can be empty. The confidence score and kept candidates help a human check these.
- Our evaluation set is small (2 fully transcribed pages). More pages are being collected.
- Next: a faster handwriting model (e.g. a GPU-backed TrOCR), and splitting each page into separate question records.

12. Contributions

> **In short**
> The work was split **equally** among the five members (20% each). Each member owned one part of the pipeline from start to finish.

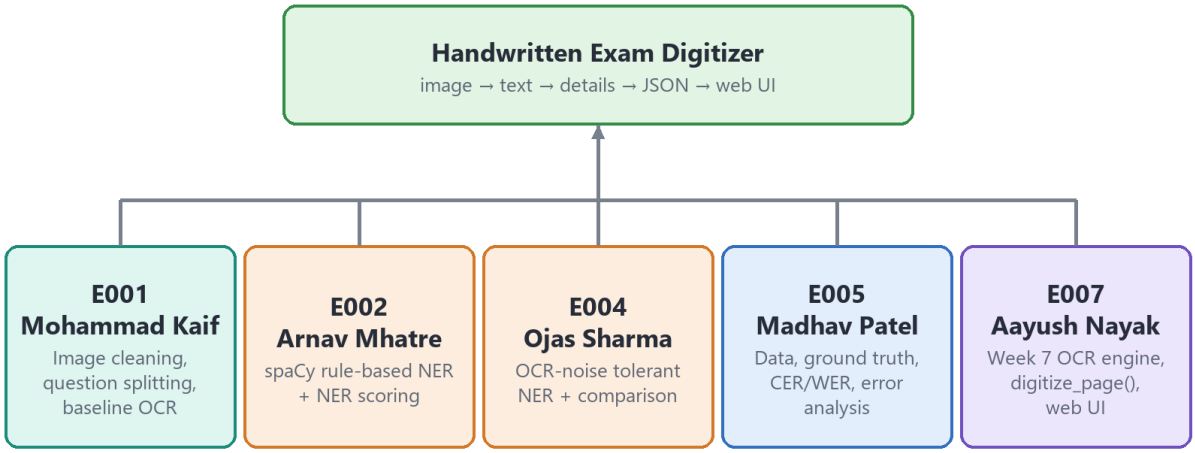


Figure 7. Who built which part

Roll No.	Name	Main area	Share
E001	Mohammad Kaif	Image cleaning, question splitting, baseline OCR (Week 1-2)	20%
E002	Arnav Mhatre	spaCy rule-based NER and NER scoring (Week 3)	20%
E004	Ojas Sharma	OCR-noise tolerant NER and comparison (Week 4)	20%
E005	Madhav Patel	Data collection, ground truth, CER/WER evaluation, error analysis (Week 5)	20%
E007	Aayush Nayak	Reusable digitize_page() asset, Week 7 OCR engine and NER fixes, web UI (Week 6-7)	20%

E001 Mohammad Kaif

> **Area**
>
> Image preparation and the first working OCR (Week 1-2).
>
> Files: `src/preprocess.py`, `src/segment.py`, `src/baseline_ocr.py`

What they did

- Wrote the first **image cleaning** step: convert to grayscale, blur slightly to remove noise, **adaptive threshold** to make it black and white, then **deskew** (straighten) the page.
- Wrote **question segmentation**: uses Tesseract's word positions and a regex to spot markers like Q1, Q2 and groups the words under each question.
- Built the **baseline OCR**: plain Tesseract reading plus an average confidence score per page.
- Set up the project folder layout (`data/`, `src/`, `results/`).

How to explain it (simple script)

"My part was the very first stage: getting from a picture to some text. I cleaned the image by turning it grey, removing noise, making it black and white and straightening it."

"Then I ran Tesseract on it to get the baseline text and a confidence score, and split the text into questions by finding Q1, Q2 and so on."

"This baseline gave the team something to measure against. Later testing showed the black-and-white step hurt phone photos, and that led to the improved cleaning in Week 7."

Points to remember

- Adaptive threshold = decide black or white for each small area separately, so uneven light is handled.
- Deskew uses `cv2.minAreaRect` to find the tilt angle and rotates it back.
- The 'Threshold (Wk 1-2)' option in the web UI still runs this code, for comparison.

If ma'am asks…

Question	Answer
Why convert to grayscale?	OCR needs light vs. dark, not colour. Grayscale is simpler and faster.
What is a baseline?	The simplest working version. We compare every improvement against it.
How does segmentation find questions?	Tesseract gives each word with its position; a regex matches words like Q1 / Q2. and starts a new group there.

E002 Arnav Mhatre

> **Area**
>
> First version of Named Entity Recognition using spaCy (Week 3).
>
> Files: `src/ner_extractor.py`, `src/process_with_ner.py`, `src/evaluate_ner.py`

What they did

- Built the **spaCy NER pipeline** using an **EntityRuler** with patterns for 4 custom labels: STUDENT_ID, QUESTION_NUM, MARKS, PAGE_NUM.
- Connected OCR and NER in `process_with_ner.py` so one page gives a record with ID, question and marks.
- Wrote the **NER evaluation**: precision, recall and F1 on labelled examples.

How to explain it (simple script)

"After we have text, we need to know who wrote it and which question it is. That is Named Entity Recognition."

"I used spaCy's EntityRuler, which lets us write patterns. For example, a student ID looks like 2 digits, a letter, 3-4 digits, and a question looks like Q followed by a number."

"I also measured it with precision, recall and F1. This version worked on clean text, which is the base the later NER was built on."

Points to remember

- EntityRuler = rule-based NER inside spaCy; no training data needed.
- Labels: STUDENT_ID, QUESTION_NUM, MARKS (like 7/10), PAGE_NUM (like 'Page 4').
- Weakness found: if OCR writes 0 as O, the pattern fails. That led to Week 4.

If ma'am asks...

Question	Answer
Why rules and not a trained model?	We had very little labelled data, and IDs/marks follow fixed formats, so rules are accurate and explainable.
What is F1?	The balance (harmonic mean) of precision and recall; 1 is perfect.
Where does spaCy's own NER fit?	The ruler runs before spaCy's built-in NER so our custom labels take priority.

E004 Ojas Sharma

> **Area**
>
> NER that still works when OCR makes mistakes (Week 4).
>
> Files: `src/ner_improved.py` (Week 4 version), `src/compare_ner_approaches.py`

What they did

- Designed the **OCR look-alike table**: O→0, l/I/L→1, S→5, B→8, Z→2, applied only inside IDs, question numbers and marks.
- Wrote **candidate finders** for each label that accept noisy text, then clean it (e.g. '23DOO1' → 23D001).
- Made the extractor **keep every candidate** and pick the earliest one per label as the best guess.
- Wrote the **comparison script** that scores the Week 3 and Week 4 approaches on the same test cases.

How to explain it (simple script)

"OCR is not perfect. It often reads a zero as the letter O or a one as a small l. The Week 3 rules then miss the student ID."

"My job was to make NER tolerant to that. I made a table of letters that look like digits and swap them back, but only inside things like IDs and marks, not normal words."

"Then I compared both approaches on the same examples to show the improvement with numbers."

Points to remember

- The fix is applied only to small runs of characters, so normal words are not damaged.
- All candidates are returned, so a teacher can review if the best guess is wrong.
- The comparison script prints precision, recall and F1 for both versions.

If ma'am asks…

Question	Answer
Why not just replace every O with 0?	That would break normal words like 'OSI'. We only fix characters inside a pattern that must be digits.
How is the best candidate chosen?	Candidates are sorted by position; the first one for each label is picked.
Why keep all candidates?	For low-confidence pages a human can quickly see the alternatives.

E005 Madhav Patel

> **Area**
>
> Data, ground truth and accuracy testing (Week 5).
>
> Files: `src/evaluate.py`, `data/ground_truth/*.json`, `data/raw_scans/`

What they did

- Collected handwritten answer-sheet photos and prepared the **ground truth** (the true typed text) in a JSON format per page.
- Wrote the **CER/WER evaluation** with the jiwer library, which runs over a folder and compares OCR with ground truth.
- Ran the pipeline on **real** pages (Weeks 1-4 used made-up text) and recorded the **error analysis** table of failure cases.
- Tested the full system and the web UI's accuracy check against known pages.

How to explain it (simple script)

"You cannot improve what you do not measure. I collected real handwritten pages and typed out exactly what is written, which we call ground truth."

"I then compared the OCR output with it using two numbers: Character Error Rate and Word Error Rate."

"Running on real pages showed the true error rate and listed where it failed, which told the team what to fix next."

Points to remember

- CER = (substitutions + deletions + insertions) / number of characters in the truth.
- Ground truth file has `student_id` and a list of `answers` with `question` and `answer_text`.
- Real numbers: average CER went from 0.426 to 0.189 after the Week 7 changes.

If ma'am asks...

Question	Answer
Why both CER and WER?	One wrong letter makes a whole word wrong, so WER is stricter. CER shows smaller progress better.
Why test on real pages?	Synthetic text hides real problems like ruled lines and shadows.
Is the test set big enough?	Not yet: 2 fully transcribed pages. More are being collected.

E007 Aayush Nayak

> **Area**
>
> Reusable asset, Week 7 accuracy improvements and the web UI (Week 6-7).
>
> Files: `src/digitizer.py`, `src/ocr_pipeline.py`, `src/ner_improved.py` (Week 7 rules), `app/`, `start.bat`, `stop.bat`

What they did

- Found **why accuracy was low**: the Week 1-2 black-and-white threshold made ruled lines stick to letters on phone photos.
- Built the **new OCR front end** (`ocr_pipeline.py`): red channel, lighting fix, straightening by ruled-line angle, ruled-line erasing, 1.5x upscale, Tesseract psm 4, text clean-up, optional spell-check. Average **CER fell from 0.426 to 0.189**.
- Tested alternatives (TrOCR, line-by-line OCR, tessdata_best) and kept the fastest accurate option (~1 second per page).
- Wrapped the whole pipeline into one reusable function, **digitize_page()**, with a clear input/output format and confidence score so Group 2 can use it.
- Added NER rules for **real OCR errors** seen on our pages: 'QL.' → Q1, '@1)' → Q1, ID after 'Roll no', and dates not read as marks (F1 0.308 → 1.0).
- Built the **web UI**: Flask server with background jobs, PDF support, five tabs, live progress, accuracy check, downloads, and start/stop scripts.
- Joined everyone's parts together and kept the shared GitHub repository up to date.

How to explain it (simple script)

"When we tested on real pages, the error rate was about 43%. I looked into why, and found that the black-and-white cleaning was making ruled lines merge with letters."

"So I built a new cleaning step: use only the red colour layer so pink lines disappear, remove shadows, straighten the page, erase the lines gently and enlarge the image. That brought the error rate down to about 19%, and to about 4% on a neat page."

"I then packaged the whole pipeline into one function, digitize_page, so any other program can call it with an image path and get a clean JSON record back."

"Finally I built the web app so anyone can upload a PDF or photo, watch each step, see what the OCR saw, and download the result."

Points to remember

- Red channel trick: red/pink lines are bright in the red channel, while blue/black ink stays dark.
- Lighting fix = divide the image by a heavily blurred copy of itself.
- Spell-check is off by default because it would also correct the student's own mistakes.
- The UI runs each upload as a background job; the browser polls `/api/jobs/<id>` for progress.

If ma'am asks…

Question	Answer
Why not use a deep learning model like TrOCR?	Similar accuracy on our pages but 3-5 minutes per page on CPU vs. about 1 second for ours.
What does ocr_confidence mean?	Tesseract's average word confidence (0-1). Low values tell the user to check the page manually.
How can another group reuse this?	Import `digitize_page` and pass an image path. The output is a fixed JSON format.
Why use Flask?	It is small, Python-based, and lets the UI call the same pipeline code directly.

13. Glossary

Term	Meaning
OCR	Optical Character Recognition: software that reads text from an image.
NER	Named Entity Recognition: finding and labelling important bits of text (IDs, numbers, names).
Tesseract	Free OCR engine. Uses an LSTM neural network to read text lines.
LSTM	A type of neural network good at reading sequences, like letters in a line.
psm	Page Segmentation Mode in Tesseract: tells it how the page is laid out.
DPI	Dots per inch: how sharp a PDF page image is. Higher = more detail, slower.
Adaptive threshold	Turning an image into pure black and white, area by area.
Deskew	Rotating a tilted page so text lines are horizontal.
Regex	A pattern language for matching text, e.g. Q followed by digits.
Ground truth	The correct text, typed by a person, used to check OCR output.
CER / WER	Character / Word Error Rate. Share of letters / words that are wrong.
Precision / Recall / F1	How many found items were right / how many real items were found / balance of both.
JSON	A simple text format for structured data, readable by any program.
API	A web address the browser calls to ask the server to do something.